

Programmable In-Network Timing Obfuscation for DNP3

Anonymous Submission

Abstract—Adversaries preparing attacks on industrial control systems (ICSs) such as power grids first perform reconnaissance, and device fingerprinting supplies much of the knowledge they need. In ICS networks, the fingerprint does not have to come from the payload. The interval between the transport acknowledgment of an outstation and its application response carries the execution time of the device, and it identifies both the device and the operation it performs. Removing that execution time is challenging. Existing traffic obfuscation pads packets and adds dummy ones inside encrypted flows, whereas ICS traffic is unencrypted, and any padding or dummy packet reveals itself.

To overcome these challenges, we propose an in-network timing-obfuscation framework that replaces the execution time of the device with a value the operator configures. A programmable switch in front of the outstation holds the acknowledgment and the response and releases each of them at a deadline computed from the arrival of the request, so neither release depends on anything the outstation does. The switch only delays the packets it receives, so both endpoints run unmodified and no DNP3 byte changes. We implement the framework on an Intel Tofino-1 switch and evaluate it with a physical SEL-751A protective relay over 63,360 DNP3 exchanges. The experimental results show that, without adding a frame or a byte, the framework removes the execution time from the interval an adversary measures. The interval settles on the configured 8 ms for every operation, and a classifier trained before deployment falls from 0.803 balanced accuracy to chance.

Index Terms—traffic obfuscation, device fingerprinting, DNP3, programmable switches.

I. INTRODUCTION

Device fingerprinting has been an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and to design effective, stealthy attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to inflict physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks. In the 2015 attack that disrupted Ukrainian power grids and the Stuxnet attack that disrupted Iranian nuclear power facilities, it is widely believed that adversaries stayed in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies have presented network traffic obfuscation [3], [4]. Device fingerprinting targeting general computing environments generally relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5]. Consequently, existing traffic obfuscation often focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3], [6], [7], and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [8]–[10]. Since manipulating communication networks can introduce runtime overhead, recent works have begun to offload traffic obfuscation onto programmable network switches, which change communication patterns at much higher line rates than CPUs [11]–[13].

Unfortunately, these methods cannot be directly applied to ICS environments for two main reasons. First, device fingerprinting methods on ICS devices rely on different features from the ones in general computing environments. Instead of using network layer knowledge, they attempt to use behavior on physical devices to reveal device model or types of operations. For example, work in [14]–[17] uses the latency between the TCP acknowledge packets and the actual response from the same device to estimate execution time there. Since each vendors may choose different materials or algorithms to implement the same control functionality, this time can be accurate in fingerprinting those devices. Second, existing traffic obfuscation is performed over encrypted channels, which are necessary to mix the obfuscated and normal traffic. Unfortunately, ICS networks still rely on unencrypted traffic due to a wide range of legacy devices, despite ICS network protocols may provide security features. Directly padding or adding dummy packets can easily reveal obfuscated packets, exposing the the underlying traffic pattern to adversaries.

In this paper, we propose an in-network timing-obfuscation framework for ICS traffic. Instead of padding packets or adding dummy ones, the framework changes only when the packets of a transaction leave the network, such that the timing an adversary measures becomes a value the operator configures rather than a property of the device.

A. Contributions

To overcome the two drawbacks above, we design a framework that runs on a programmable switch placed in front of an outstation. The switch holds the transport acknowledgment and the application response of the outstation, and it releases each of them at a deadline computed from the moment the request

arrives. Because both deadlines are computed from the request, neither release depends on when the outstation acknowledges or answers, and the execution time of the device does not reach the intervals the adversary measures. Because the switch forwards the same packets it received, the obfuscated traffic carries nothing that marks it as obfuscated, which is what a plaintext protocol requires.

The Framework's Role. We position the framework as a complement to the protections an ICS already runs, such as intrusion detection. The framework cannot hide which operation an exchange carries, because DNP3 leaves the function code readable, and it adds a bounded delay to every exchange it protects. However, it reads no payload and changes no byte, and it requires no modification of the master, the outstation or the DNP3 protocol. Consequently, it can protect legacy devices that can neither be patched nor replaced, which are the devices that fingerprinting targets.

Even though we present the framework for DNP3, the design concept can be extended to other ICS protocols. The schedule refers only to the arrival of a request and to the two packets that answer it, and not to the content of any packet. By adding the function codes of another request-response protocol over TCP to the classification table, the switch can hold its packets in the same way.

The main contributions of the paper are:

- **Replace the execution time of a device with a configured value.** We design a release schedule that holds real protocol packets and computes both release deadlines from the arrival of the request. As such, the switch rather than the outstation sets both intervals the adversary measures, the request-to-acknowledgment interval and the cross-layer response time.
- **Obfuscate plaintext ICS traffic without adding a packet.** The framework only delays packets, so it needs no encryption to hide its operation. The operator bounds the added delay by choosing the release budget, and that bound keeps every exchange within the timers of TCP and DNP3.
- **Implement the framework at line rate.** On an Intel Tofino-1 switch, we hold each packet behind a strict-priority queue of blocker packets the switch generates. The queue sustains a hold until its deadline without a timer. We cover both the read and the control transaction of DNP3.

For evaluation, we placed the switch between a DNP3 master and a physical SEL-751A feeder protection relay and collected 63,360 exchanges in 22 grouped runs, half of them with the framework running. We measured the released interval directly, and we tested it against (i) a classifier trained before the framework was deployed, (ii) an adaptive adversary that retrain on obfuscated traffic, and (iii) an adaptive adversary that pools many exchanges of the same operation into one signature, as Formby et al. do.

Our experimental results show that the framework removes the execution time of the device from the interval an adversary measures while introducing negligible overhead:

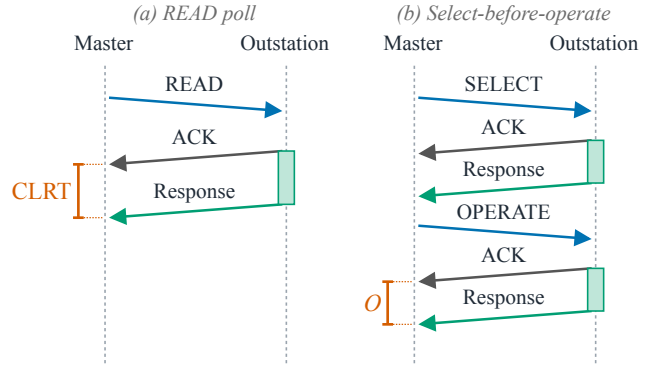


Fig. 1. The two DNP3 transactions on the master-facing link. (a) A READ poll and its CLRT. (b) A select-before-operate control and its master-visible interval O .

- The released interval settles on the configured value. The medians of READ, SELECT and OPERATE move from 2.121, 2.041 and 2.929 ms to 8.005, 8.005 and 8.000 ms, and the interquartile range of READ falls from 2.772 to 0.022 ms. The operator can place the interval anywhere from 2 to 26 ms within a fixed budget, and the switch delivers each setting to within 6 μ s.
- The interval no longer carries the execution time. READ and SELECT differ only in how long the relay takes to answer them, and a classifier that separates them at 0.785 balanced accuracy under Timing OFF is at 0.507 under the framework, against a chance level of 0.5. A classifier trained before deployment falls from 0.803 to 0.333 on the three operations, i.e., chance. An adaptive adversary reaches 0.445, and we show that it learns when requests reach the switch rather than how the relay executes them.
- The overhead is negligible. The framework adds no frame and no byte, and the median request-to-response time of a poll rises from 2.681 to 28.107 ms, well within the 200.8 ms after which the master repeats an unacknowledged request. No exchange was retransmitted.

II. BACKGROUND AND MOTIVATION

In this section, we first present the DNP3 transactions the framework protects. Then we explain why the timing of these transactions identifies a device and how a programmable switch can change that timing.

A. DNP3 Read and Control Transactions

DNP3 carries supervisory control and data acquisition traffic over TCP between a master and an outstation [18]. In Figure 1, we show the two transactions the framework protects. The master polls the outstation with a READ request, function code 1, and the outstation returns the requested measurements in a response, function code 0x81. A control command instead uses select-before-operate. The master sends a SELECT, function code 3, that names an output point, and the outstation confirms it. The master then sends an OPERATE, function code 4,

which the outstation executes and confirms with a solicited response.

Because DNP3 runs over TCP, the transport layer of the outstation acknowledges each request. The acknowledgment travels in a segment of its own before the application answers, although TCP also allows it to ride on the response [19]. Each exchange appears on the wire as three packets: a request, a transport acknowledgment carrying no DNP3 payload, and a DNP3 response. We limit the framework to the three function codes above, READ, SELECT and OPERATE, and it forwards every other exchange unchanged.

Definition 1. Cross-Layer Response Time (CLRT): the interval between the arrival of the transport acknowledgment and the arrival of the DNP3 response of the same exchange, measured on the link to the master.

Definition 2. Physical Operation Time: for a select-before-operate command, the interval between the OPERATE command and the moment the commanded physical change occurs at the device. Formby et al. estimate it in two ways, from the arrival of the unsolicited event response at a tap and from the sequence-of-events timestamp the response carries in its payload [14]. They report that the arrival-time estimate produced no usable results and retain the payload timestamp as the physical fingerprint.

B. Execution Time as a Fingerprint

An outstation answers a request twice, from two different layers. Its TCP stack acknowledges the request as soon as it arrives, whereas its application responds only after the device has done the requested work. Because both packets travel the same path in the same direction, the common propagation delay cancels. Consequently, the CLRT is dominated by the execution time of the outstation.

Execution time differs between device models, since vendors implement the same function with different hardware and algorithms, and it differs between the operations one device performs. Formby et al. classify DNP3 device types and models by the distribution of the CLRT [14], and Lin et al. measured the execution times of three devices, among them the SEL-751A we evaluate, and separated them by the combination of their average and their variation [20]. The same separation holds between the operations of one device. For example, the median CLRT of our relay is 2.929 ms for a control command and 2.121 ms for a poll, while its median acknowledgment latencies for READ, SELECT and OPERATE lie within 0.003 ms of one another. Consequently, the CLRT carries how this device performs an operation, which is the profile a fingerprinting adversary builds.

C. Holding Packets in a Programmable Switch

A programmable switch exposes a match-action pipeline, written in P4 [21], that parses headers, keeps state in registers and places packets into output queues at line rate. Each egress port offers several queues served in strict priority, a primitive that can also approximate programmable scheduling [22]. Because such a pipeline can delay a packet and release it later

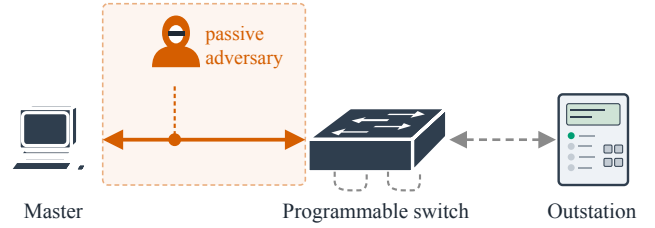


Fig. 2. Observation model. The adversary monitors the shaded master-facing link. The dashed paths are not visible to it.

without a host in the loop, a switch between the master and the outstation can hold the acknowledgment and the response of a transaction and release them on a schedule the operator sets. As such, the switch can change the timing an adversary observes without changing either endpoint or any byte of the payload.

III. THREAT MODEL AND OBJECTIVES

In this section, we first describe the adversary we consider and the knowledge it seeks. Then we specify the three research objectives the framework must achieve, RO1 to RO3, and the scope of our work.

A. Threat Model

Assumptions on Adversary's Capability. We assume that an adversary has penetrated the control network and passively monitors the master-facing link, the position from which Formby et al. fingerprint outstations [14], as shown in Figure 2. The adversary is able to record the headers, the size and the arrival time of every packet. Because DNP3 carries no transport security by default [23], the adversary also reads the function code of every request and can label each exchange by its operation, e.g., a READ poll or the SELECT and OPERATE of a control command. We assume that the adversary knows the framework and the interval it enforces, but cannot read the internal state of the switch or monitor the link between the switch and the outstation. This is a practical assumption, as the switch sits at the edge of the outstation, and the relay-facing link can be a single cable between two devices in the same substation cabinet.

Adversary's Goal. The adversary aims at fingerprinting the outstation from the timing of its exchanges, and from that timing it infers the make and model of the device and the operation each exchange performs. Based on this knowledge, the adversary can design targeted attacks, e.g., false data injection that must match the devices it compromises [24]. We consider two adversaries. A *fixed* adversary builds its classifier from traffic captured before the framework is deployed and applies it afterwards. An *adaptive* adversary collects traffic after deployment, labels it with the plaintext function codes and retrains its classifier. The adaptive adversary can also pool many exchanges of the same operation into one signature, as Formby et al. do [14].

B. Research Objectives

The Framework’s Objective. We aim at disrupting timing-based fingerprinting of an outstation without changing the outstation, the master or the DNP3 protocol. We specify three research objectives (RO) as follows:

- **RO1:** for both the read poll and the select-before-operate control, we aim at making the CLRT the adversary measures a value the operator configures, independent of the execution time of the outstation.
- **RO2:** for fixed and adaptive adversaries, we aim at removing the execution time of the outstation from everything they learn from the timing of an exchange, such that a fixed adversary performs no better than chance and whatever an adaptive adversary still learns reflects the switch rather than the device.
- **RO3:** for the operation of the control network, we aim at negligible overhead. The framework adds no packet and no byte, and it keeps every added delay within the timers of TCP and DNP3, such that no request is retransmitted and no control operation is refused.

Trusted Components. To achieve these objectives, we add only the switch and its control plane to the trusted computing base. The control plane installs the release policy and reads it back, and the switch enforces it. The framework needs no cooperation from the master or the outstation, and neither of them is modified.

For Payload-Based Fingerprinting. We designed the framework to hide timing, not content. It forwards every packet at its original size and leaves the function codes readable, so the adversary still knows which operation an exchange carries. What the framework removes is how long the device takes to perform it. The physical operation time of Definition 2 is a second feature, and the estimate Formby et al. retain reads a timestamp inside the payload rather than the timing of a packet [14]. Such content is a different problem, and payload protection is complementary to the framework. We leave the combination of the two to future work.

For Active Adversaries. An adversary that injects, delays or drops packets, or that compromises the switch or the outstation, performs attacks beyond reconnaissance and is outside this model. Such an adversary still faces existing defenses, e.g., specification-based intrusion detection for DNP3 [25], and the framework leaves every byte those defenses inspect unchanged.

IV. DESIGN

In this section, we first present an overview of the framework. Then we describe how it replaces the CLRT of a read transaction (Section IV-B) and of a control transaction (Section IV-C), each followed by its security argument. Last, we argue why its overhead is negligible and bounded (Section IV-D).

A. Design Overview

In Figure 3, we present the design of the framework. We place a programmable switch between the master and the

outstation, and the switch classifies every DNP3 exchange by its function code. Based on that classification, it holds the packets that carry the timing of the outstation and releases each of them at a deadline, such that the interval the adversary measures becomes the value the operator configures. We handle two kinds of transactions in two lanes.

Holding a Packet. A switch pipeline carries no timer a program can wait on, so the framework holds a packet with the queues instead. Every packet it delays waits in a queue of its own, beneath a second queue of higher priority that carries *blocker packets*, which the switch generates itself. While blocker packets keep arriving, the scheduler serves them and never serves the packet beneath them. The hold ends when the blocker queue runs dry at the deadline, and the held packet leaves on the next service. Blocker packets circulate on a loopback port inside the switch, so they reach neither the master nor the outstation and consume no bandwidth on either link. Section V describes how the program generates them and how it decides when they stop.

Read Lane. For a READ poll and for the SELECT phase of a control command, the framework holds the transport acknowledgment and the application response of the outstation on their way to the master. It computes both release deadlines when the request arrives (Section IV-B).

Control Lane. For the OPERATE phase, the framework also holds the command itself on its way to the relay, and it computes the deadlines of the acknowledgment and the response from the arrival of the OPERATE request with the offsets of the read lane (Section IV-C).

Both lanes forward the packets they receive at their original size and generate none on either link. The master and the outstation exchange exactly the packets they would exchange without the framework, and only the instants at which those packets leave the switch differ.

B. Replacing the CLRT of a Read Transaction

Timing Model. In Figure 4, we show the timeline of one READ transaction. The request reaches the switch at t_0 , and the outstation answers it twice: its transport layer first acknowledges the request, and its application later returns the response. We write t_A and t_R for the instants the acknowledgment and the response reach the switch, and e_A and e_R for the instants the switch forwards them. Without the framework, the adversary measures

$$\text{CLRT}_{\text{original}} = t_R - t_A, \quad (1)$$

which is dominated by the execution time of the outstation (Section II). With the framework, the adversary measures

$$\text{CLRT}_{\text{new}} = e_R - e_A = \text{CLRT}_{\text{original}} + (e_R - t_R) - (e_A - t_A), \quad (2)$$

where $e_A - t_A$ and $e_R - t_R$ are the *acknowledgment hold* and the *response hold*, the times the two packets spend in the switch.

Constant Holds. Equation (2) shows why holding both packets for constant times cannot remove the fingerprint. If

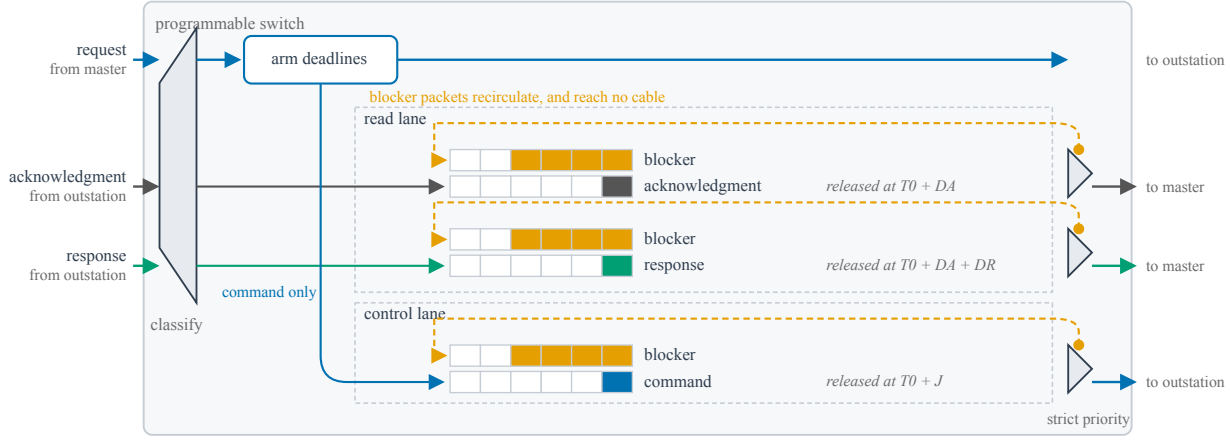


Fig. 3. The framework on one switch pipeline, read left to right. A request arms both release deadlines from its own arrival, so neither instant carries anything the outstation did. The scheduler drains each blocker queue before the queue it guards, which is what holds a packet until its deadline. The figure does not show the exchanges the switch forwards unchanged.

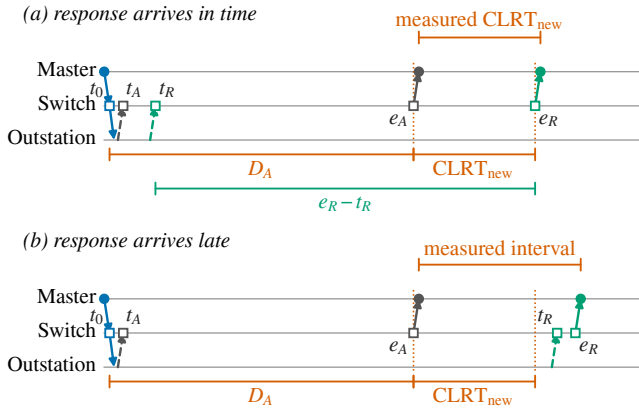


Fig. 4. Release timeline of one READ transaction. The five instants are all at the switch: the request arrives at t_0 , the acknowledgment and the response of the outstation at t_A and t_R , and each of them is released at e_A and e_R . Both deadlines are armed from t_0 , so neither depends on what the outstation did. (a) The response arrives in time. (b) It arrives late and leaves when its queue next serves it.

both holds are constant, the adversary measures the original interval shifted by their difference. The distribution moves along the time axis and keeps its shape, e.g., a standard deviation of 2.889 ms for READ on our relay, so an adversary who has profiled the device still recognizes it. To remove the fingerprint, the holds must absorb the variation of the outstation.

Dual-Deadline Schedule. To make the holds absorb that variation, we compute both release deadlines from the arrival of the request. When the request arrives at t_0 , the framework sets the scheduled instants

$$\hat{e}_A = t_0 + D_A, \quad \hat{e}_R = t_0 + D_A + \text{CLRT}_{\text{new}}, \quad (3)$$

where D_A is the *acknowledgment deadline* and the configured CLRT_{new} is the interval the operator wants the adversary to

measure. Neither deadline depends on t_A or t_R , so the two holds become

$$e_A - t_A = D_A - (t_A - t_0), \quad e_R - t_R = D_A + \text{CLRT}_{\text{new}} - (t_R - t_0). \quad (4)$$

Both holds follow the device on every transaction. Each is long when the outstation answers quickly and short when it answers slowly.

We anchor both deadlines at the request rather than at the acknowledgment for two reasons. First, the acknowledgment is sent by the outstation, so a deadline measured from it would carry the acknowledgment latency of the outstation into the request-to-acknowledgment interval, which the adversary also measures. Second, the control lane must time its deadlines from the request (Section IV-C). With one reference for both lanes, the lanes differ only in the command they hold, and the adversary cannot use the reference itself to tell them apart.

Availability. The switch can delay a packet but cannot release one it has not received. We call

$$D = D_A + \text{CLRT}_{\text{new}} \quad (5)$$

the *release budget*, and the schedule requires

$$t_R \leq t_0 + D, \quad (6)$$

i.e., the budget must cover the whole time the outstation takes to answer a request. A response that violates (6) arrives after its deadline. The framework marks it late and forwards it as soon as its queue serves it, so the exchange completes with its own timing. The operator chooses D such that late responses are rare (Section IV-D).

Security Argument. Each held packet leaves the switch slightly after its deadline, by a *release tail* ε that the queues of the switch introduce (Section V). Writing $e_A = \hat{e}_A + \varepsilon_A$ and $e_R = \hat{e}_R + \varepsilon_R$, the schedule (3) gives, for every response that satisfies (6),

$$e_A - t_0 = D_A + \varepsilon_A, \quad e_R - e_A = \text{CLRT}_{\text{new}} + \varepsilon_R - \varepsilon_A. \quad (7)$$

No term on the right-hand side depends on t_A or t_R . Consequently, the execution time of the outstation reaches neither the request-to-acknowledgment interval nor the CLRT the adversary measures. The operator sets the configured values, and the release tails belong to the switch: a tail is the time a blocker queue takes to drain after its deadline, which depends on the state of the queue when the request arrives and not on the device. Nothing in this argument refers to which operation the outstation performed or which device produced the response. A different operation and a different device type are both a different distribution of $t_A - t_0$ and $t_R - t_0$, and both are absorbed in the same way by (4). Therefore, a classifier that separates device types or operation types by these intervals, such as the one of Formby et al. [14], receives intervals set by the switch rather than by the device. Pooling many exchanges cannot recover what the intervals do not contain. The device can still reach the intervals in only one way, through responses that violate (6), and we evaluate how often this happens in Section VI.

Is It Possible to Recover the Execution Time (Adaptive Adversary)? An adaptive adversary knows the framework and the configured values, but this knowledge reveals only values the operator chose. Retraining on obfuscated traffic gives it the intervals of (7), which contain the configured offsets and the release tails. A release tail depends on when a request reaches the blocker queue, and the master decides when each request is sent, e.g., it sends an OPERATE immediately after the response to its SELECT. Consequently, an adaptive adversary can learn from the tails when requests reach the switch, which it can already read from the request timestamps, but not how long the outstation takes to execute them. We quantify what it learns in Section VI-A2.

C. Replacing the CLRT of a Control Transaction

A select-before-operate control command differs from a poll in one way. The framework also holds the OPERATE request before it reaches the relay, which separates the instant the master issues the command from the instant the relay acts on it. When the OPERATE arrives at T_0 , the framework releases it toward the relay after a command hold J drawn from a configured set, and it schedules the acknowledgment and the response toward the master at

$$\hat{e}_{\text{ack}} = T_0 + D_A, \quad \hat{e}_{\text{or}} = T_0 + D_A + \text{CLRT}_{\text{new}}. \quad (8)$$

The acknowledgment of the relay exists only after the command is released, so a deadline measured from it would carry the command hold into the interval the adversary measures; the request is the only reference that precedes both. The reference and the offsets are those of the read lane, and one set of deadline registers serves both lanes (Section V).

Security Argument. The adversary measures $O = e_{\text{or}} - e_{\text{ack}} = \text{CLRT}_{\text{new}} + \varepsilon_{\text{or}} - \varepsilon_{\text{ack}}$, in which neither the execution time of the relay nor the command hold J appears. As such, the framework can vary the instant the relay acts on a command while the interval the adversary measures stays fixed, and the adversary cannot subtract a known offset to

recover either quantity. As in the read lane, the schedule requires the response of the relay to reach the switch before $\hat{e}_{\text{or}}$, i.e., within D of the request, with the command hold J included.

D. Overhead Argument

Obfuscation is only useful if an operator can afford it. We now argue why the framework introduces negligible overhead (RO3) in any deployment.

Bandwidth Overhead. As shown in Figure 3, the framework forwards the packets it receives at their original size. The blocker packets that sustain a hold recirculate on internal loopback ports and never reach a cable (Section V). Consequently, the framework adds no frame and no byte to either link. Obfuscation that pads packets or injects dummy ones pays a bandwidth cost that this design does not have [4], [11].

Latency Bound. We delay two packets per transaction. The acknowledgment leaves at $t_0 + D_A + \varepsilon_A$. A response that satisfies (6) leaves at $t_0 + D + \varepsilon_R$, so its added delay, $D - (t_R - t_0) + \varepsilon_R$, never exceeds $D + \varepsilon_R$ and shrinks as the outstation answers more slowly. A response that violates (6) leaves as soon as its queue serves it. The operator therefore bounds every added delay by choosing the release budget D , and the release tail is the only term the operator does not set.

Protocol Timers. Three timers of the two endpoints can observe these delays. The TCP stack of the master retransmits a request whose acknowledgment does not arrive within its retransmission timeout τ_m . The TCP stack of the outstation retransmits a response the master does not acknowledge within its own timeout τ_o . The relay cancels a selected control point whose OPERATE does not follow within the select-to-operate timeout τ_s , 1.0 s by default on the SEL-751A [26]. The acknowledgment hold delays the first, the response hold delays the second, and the holds on the SELECT response and on the OPERATE command delay the third. Consequently, the framework triggers none of them as long as

$$D + J_{\text{max}} + \varepsilon < \min(\tau_m, \tau_o, \tau_s), \quad (9)$$

where J_{max} is the largest command hold. Condition (9) holds for any deployment whose outstation answers well within its timers, because the budget only needs to cover the response time of the outstation. RFC 6298 recommends a lower bound of one second on the retransmission timeout [27]. On our testbed, the master repeats an unacknowledged request after 200.8 ms and the relay repeats an unacknowledged response after 2.969 s, while $D + J_{\text{max}}$ is 40 ms and a 28 ms budget covers 99.94% of the read-lane responses of the relay (Section VI-B).

Security-Performance Trade-off Argument. The release budget D sets both sides of a trade-off. A larger budget places more of the responses of the outstation on the schedule, so fewer late responses carry their own timing, and it adds more delay to every transaction. Within a fixed budget, the split between D_A and the configured CLRT_{new} costs nothing, and the operator can place the released interval anywhere below D without changing the delay. We set CLRT_{new} to a constant,

because a constant removes the variation of the device entirely and an operator can state and check it. The schedule can carry any other value, e.g., one drawn from a distribution, without change.

V. IMPLEMENTATION

To evaluate the framework, we implemented it on an Intel Tofino-1 switch and placed the switch between a DNP3 master and a physical protective relay. In this section, we first describe how the switch holds a packet until its deadline. Then we describe how it processes a transaction and the testbed we use.

A. Holding Packets with Queues

A P4 program runs to completion on each packet, and it can read a clock. However, it can neither sleep nor schedule a callback, so it cannot wait for a deadline by itself. The switch does offer several queues per port and a strict-priority scheduler [22]. Because a scheduler decides which packets do not leave, we hold a packet with the scheduler rather than with a timer.

Blocker Queues. As shown in Figure 3, the read lane uses four queues on one internal port. Two queues carry the packets we delay, the acknowledgment and the response. Above each of them sits a *blocker queue* of higher priority, which carries small packets the switch generates itself. While blocker packets keep arriving, the packet beneath them is never served, and the hold ends when its blocker queue runs dry. The response has its own held queue and its own blocker queue, so the switch can release the acknowledgment and the response independently. Blocker packets recirculate on an internal loopback port and never reach a cable, so neither endpoint can observe them. Each blocker packet carries a tag naming its transaction and lane, so a leftover blocker packet cannot delay a later transaction.

Deadline Check. No counter drives the hold. When the request arrives, the program stores an absolute release instant for each packet, quantized to a 256 ns grid. Each of the 64 recirculating blocker packets compares the current timestamp against that instant on every pass. The program regenerates the blocker packet while the deadline lies in the future and drops it once the deadline passes. Consequently, the deadline decides when the held packet leaves, and the blocker packets only keep the queue non-empty until then.

Release Tail. The held packet does not leave at the deadline itself. Blocker packets already queued when the deadline passes are served once more before the queue runs dry, and only then does the held packet leave. This drain time is the release tail ε of Section IV-B, and we measure it in Section VI-B2.

B. Processing a Transaction

Read Transaction. When a request arrives, a table classifies it by its function code, and the program forwards it at once, records the transaction in flight and writes both release instants from the arrival timestamp of the request, as (3) requires.

It writes them only for a fresh transaction, and the later acknowledgment of the outstation finds both deadlines already armed and cannot move them. The acknowledgment and the response each wait in their held queue and leave at their scheduled instants. A response that arrives after its deadline is marked late and enters the same held queue, from which it leaves when the queue next serves it. Releasing the response clears the transaction.

Control Transaction. The control lane uses a second loopback port with one blocker queue above the held OPERATE request, on its way to the relay. The acknowledgment and the response of the OPERATE then pass through the read-lane queues, which release them at the instants of (8). Placing the OPERATE on its own loopback port gives it a scheduling domain of its own, so the higher-priority read-lane queues cannot starve it. Both lanes share one set of deadline registers, so the program protects one transaction at a time. A second transaction that would arm while one is in flight is forwarded unprotected rather than disturbing the deadline in progress.

Failing Open. Every path through the program ends in a forwarding decision. A packet outside the protected session, an unrecognized request, an acknowledgment without an armed transaction and a response whose transaction has already been released all leave unchanged. A budget on blocker passes bounds every hold, so once a blocker queue spends its budget the held packet leaves and its state clears. From this budget, the control plane derives the longest hold the switch can sustain, a horizon H of 30.8 ms. With the framework disabled, the same program forwards every packet at once, which gives the Timing OFF case.

C. Testbed

Switch Program. The program has 1,584 lines of P4₁₆ [21] for the Tofino Native Architecture (comments excluded), and we compiled it with the Barefoot SDE 9.13.2. It occupies all twelve ingress stages and six egress stages of one pipeline, with 113 tables. Its state lives in one-entry registers, each accessed once per packet. One 25 Gb/s port connects the master and one 1 Gb/s port connects the relay. Two further ports are loopbacks without cables, one for the read lane and one for the control lane.

Control Plane. A control plane written in Python, around 900 lines in its entry module, brings up the ports and the queues, configures the packet generator, and writes the release policy into runtime tables and reads it back. An operator therefore changes the policy by writing table entries rather than loading a new program. The control plane refuses any release offset at or above the smallest of the timer and horizon bounds it is given, which is 30 ms on our testbed.

Physical Device. The outstation is an SEL-751A feeder protection relay, one of the DNP3 device classes that Formby et al. fingerprint [14].

DNP3 Master. The master issues READ polls and select-before-operate controls through a guarded driver. Every control operation addresses an isolated control point, so no operation in the evaluation moves a breaker.

VI. EVALUATION

In this section, we perform security and performance evaluation of the framework. The security evaluation covers RO1 and RO2 (Section VI-A), and the performance evaluation covers RO3 (Section VI-B).

Evaluation Cases. We compare two cases on the testbed of Section V. In *Timing OFF*, the switch forwards the acknowledgment and the response of the outstation as they arrive. In *Obfuscated*, the framework holds and releases them on the schedule of Section IV. Both cases run the same switch program and differ only in the release schedule. We timestamp every packet on the master-facing link, where the adversary sits, so every interval we report is one the adversary can measure.

Datasets. We use two datasets. In the main campaign, we collected 63,360 DNP3 exchanges in 132 captures and 22 grouped runs, half in each case. We interleaved the two cases over 5.3 hours so that drift cannot separate them. We set D_A to 20 ms and the configured CLRT_{new} to 8 ms, which gives a release budget D of 28 ms, and the control lane draws J from $\{2, 6, 12\}$ ms. A policy sweep installs 26 settings, 23 release policies and three controls, with 9,360 exchanges in total, and a readback from the control plane confirms each installed setting. Every request in the main campaign was answered, and no exchange was excluded.

Adversary Models. Following Formby et al. [14], the adversary classifies each exchange into one of three operations, READ, SELECT and OPERATE, from its CLRT and its request-to-acknowledgment interval, with a random forest. We report balanced accuracy, for which chance is 0.333, and hold out one grouped run at a time. We also estimate the mutual information between the CLRT and the operation against a null built by shuffling the labels within each run 1,000 times, which measures the information available to any classifier.

A. Security Evaluation

Security evaluation focuses on (i) whether the framework replaces the timing of the outstation with the configured value (RO1), and (ii) what fixed and adaptive adversaries still learn (RO2).

1) *Effectiveness of Timing Replacement: Effectiveness in RO1.* In Figure 5, we show the READ CLRT the adversary measures in both cases. Under Timing OFF, the interval spans 1.007 to 113.480 ms and reflects how long the relay takes to answer. Under the framework, 99.94% of READ exchanges fall within 0.10 ms of the configured 8 ms. From the figure, we can see that the medians move from 2.121 ms for READ and 2.041 ms for SELECT to 8.005 ms for both, while the interquartile ranges fall from 2.772 and 2.679 ms to 0.022 and 0.021 ms. A defense that only shifted the interval would keep the Timing OFF spread, since shifting a distribution leaves its variance unchanged. Shifting the Timing OFF READ intervals by a constant keeps their standard deviation of 2.889 ms, whereas the framework reduces it to 0.989 ms for READ and from 2.267 to 0.017 ms for SELECT. The framework therefore replaces the interval rather than moving it. In 21 of the 22

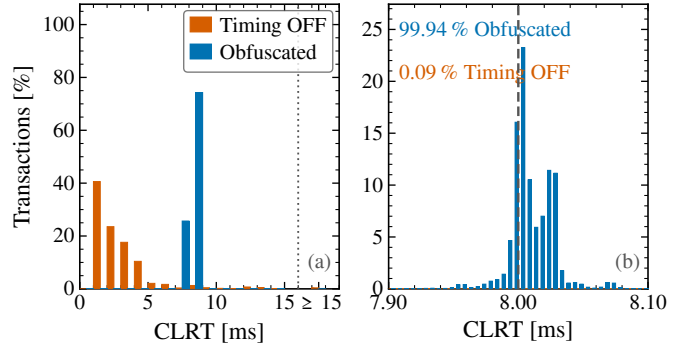


Fig. 5. READ CLRT on the master-facing link, as a percentage of each case’s own transactions. (a) 1 ms bins, with everything at or above 15 ms in the category on the right. (b) 0.005 ms bins within 0.10 ms of the configured value, dashed; the percentages name the share of each case that falls inside this window.

runs, the variance of the obfuscated READ interval lies below that of every Timing OFF run, and the remaining run contains responses that arrived after their deadline.

Removing the Execution Time. READ and SELECT follow the same path through the switch and differ only in how long the relay takes to answer them. Under Timing OFF, a classifier trained on the two separates them at 0.785 balanced accuracy from both intervals, against a chance level of 0.5. Under the framework, the same classifier, trained on obfuscated traffic, reaches 0.507, and the medians of the two classes are identical, 8.005 ms for the CLRT and 20.101 ms for the request-to-acknowledgment interval. This is because the framework removes the execution time from both intervals (Section IV-B), so nothing remains that separates two operations that differ only in execution time.

Control Lane. The control lane achieves the same replacement for OPERATE, although the framework holds the command itself for a varying J before it reaches the relay. The OPERATE median moves from 2.929 ms under Timing OFF to 8.000 ms, and its interquartile range falls from 2.729 to 0.006 ms. As (8) predicts, J does not appear in the interval.

Programmable Policy. Based on one budget, an operator can place the released interval anywhere within it. In Figure 6(b), we show that at a fixed budget of 28 ms, configured values from 2 to 26 ms produce measured medians within 6 μ s of the setting, while the median request-to-response time stays between 28.106 and 28.108 ms. Consequently, the operator sets the interval the adversary measures and the delay of the exchange independently. The sweep also shows the upper end of the range. The acknowledgment follows its deadline up to 30 ms, where the median request-to-acknowledgment interval is 30.107 ms and the released interval still sits at 3.999 ms, its configured 4 ms. Beyond that setting the switch can no longer sustain the hold, the acknowledgment leaves at 31.07 ms whatever the setting, and the released interval grows to 9.034 ms at a 36 ms setting.

2) *Effectiveness against Fingerprinting: Effectiveness in RO2.* Under Timing OFF, the CLRT separates the three

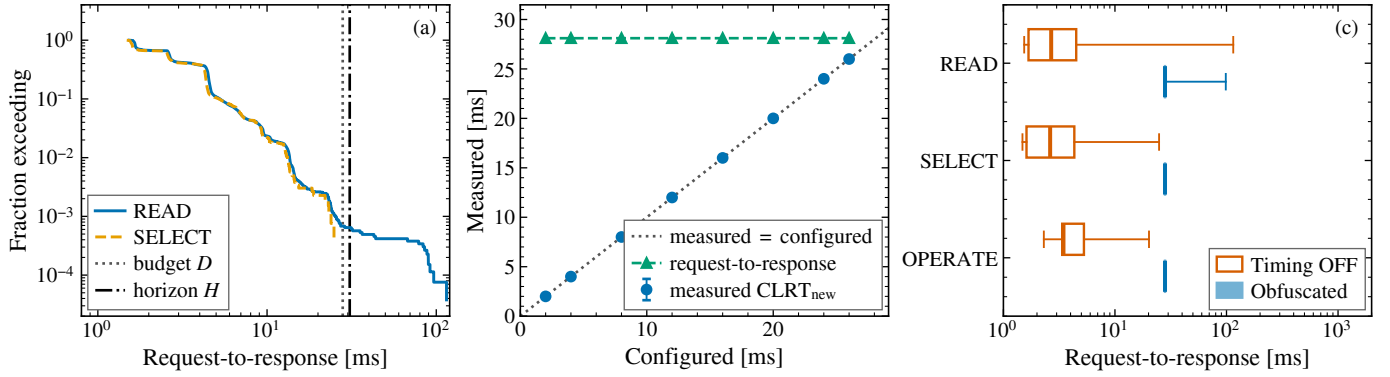


Fig. 6. Release policy and cost. (a) The Timing OFF request-to-response time the budget must cover, for the read lane only. (b) Measured against configured interval at a fixed budget $D = 28$ ms. (c) Request-to-response time per operation, in both cases.

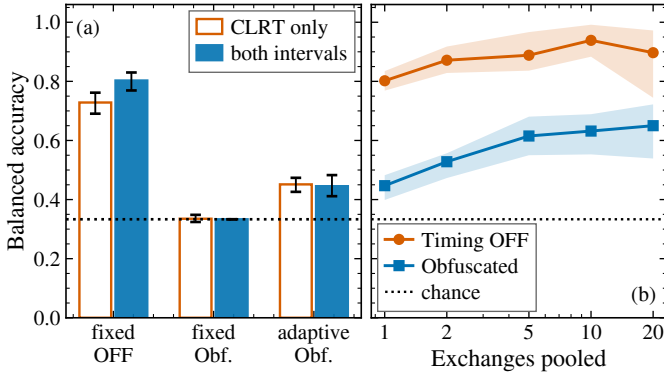


Fig. 7. Fingerprinting the three operations, one grouped run held out at a time. (a) Balanced accuracy by adversary; the fixed adversary is trained on Timing OFF traffic and applied unchanged, the adaptive one is retrained on obfuscated traffic. (b) What pooling several exchanges of one operation buys the adaptive adversary, from both intervals. Whiskers and bands span the 22 runs, which is within-campaign variability and not a confidence interval.

operations, with medians of 2.121, 2.041 and 2.929 ms, while their request-to-acknowledgment medians of 0.553, 0.551 and 0.554 ms barely differ. In Figure 7(a), we show that a fixed adversary trained on Timing OFF traffic reaches 0.728 balanced accuracy from the CLRT alone and 0.803 from both intervals. Applied to obfuscated traffic, the same adversary falls to 0.335 and 0.333, i.e., chance. The mutual information between the CLRT and the operation falls from 0.353 to 0.043 bits. The remaining information lies above its permutation null ($p = 0.001$), and the adaptive adversary below shows where it comes from.

Adaptive Adversary. An adaptive adversary retrained on obfuscated traffic reaches 0.451 from the CLRT alone and 0.445 from both intervals. In Figure 7(b), we show that pooling helps it, as it does under Timing OFF. When it pools twenty exchanges of both intervals, its accuracy rises to 0.650, against 0.897 under Timing OFF. Its accuracy comes almost entirely from OPERATE, which it recognizes in 0.689 of cases, while it recognizes READ in 0.411 and SELECT in 0.235 of cases. We find that this accuracy reflects when requests reach the switch rather than how the relay executes them, for three reasons.

First, READ and SELECT, which differ in execution time, are inseparable (Section VI-A1). Second, READ exchanges, which share one execution time, become separable when we split them by the gap that preceded each request. The two halves differ by 31 μ s in their median gap, and the same classifier separates them at 0.628 under the framework, against 0.536 under Timing OFF. Third, OPERATE reaches the switch in a different way from the other two operations. The master sends it 0.354 ms after the response to its SELECT, whereas a READ follows the previous exchange by 20.505 ms, and the release tail of OPERATE has a median of 107 μ s, against 101 μ s for READ and SELECT. As argued in Section IV-B, the arrival of a request sets the state of the blocker queue it meets, and the master, not the outstation, decides when each request arrives.

Discussion. Operations of one device differ in execution time, e.g., by 0.8 ms between a poll and a control command on our relay, and device types differ in the same way [14], [20]. Because the framework removes the execution time whatever its source (Section IV-B), the same argument applies to device types. Behind the switch, every outstation whose responses arrive within the budget presents intervals set by the same schedule. What an adaptive adversary still learns lies in the release logic of the switch and in the schedule of the master, both of which the operator controls.

B. Performance Evaluation

The framework introduces overhead in two forms, bandwidth and latency, and the latency can interact with the timers of TCP and DNP3. We evaluate each against RO3.

1) Bandwidth and Latency Overhead: Effectiveness in RO3. We observe no bandwidth overhead. Each of the 132 captures carries 1,448 frames and 130,708 bytes on the master-facing link in both cases, i.e., 3.017 frames and 272.308 bytes per exchange, so the framework moves packets in time without adding a frame or a byte. In Figure 6(c), we show the request-to-response time, whose median the framework raises by 25.426 ms for READ, 25.482 ms for SELECT and 24.652 ms for OPERATE. Each increase stays below the 28 ms

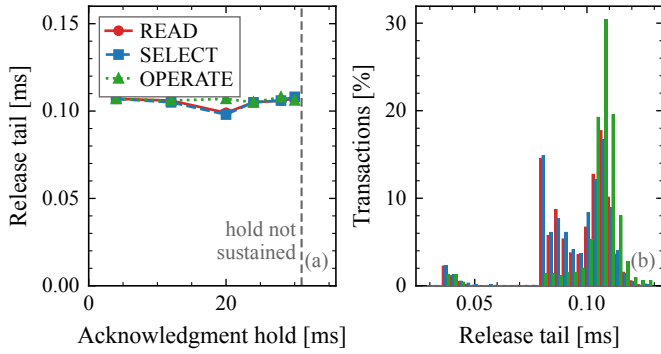


Fig. 8. Release tail ε . (a) Against the acknowledgment deadline, one series per operation. Past the dashed line the acknowledgment leaves at 31.07 ms whatever the setting, so the deadline is not met and no tail is defined. (b) Its distribution at the campaign policy.

budget, as the latency bound of Section IV-D predicts, because the holds absorb the response time of the outstation.

2) *Release Accuracy: Release Tail.* Each held packet leaves after its deadline by the release tail ε . Because the switch timestamps no departure, we obtain ε on the master-facing link, as the request-to-acknowledgment interval minus the configured D_A , which also contains the path between the master and the switch. In the main campaign, the median is 0.101 ms for READ and SELECT and 0.107 ms for OPERATE, and 99% of all values lie between 36 and 127 μ s. In Figure 8, we show that the tail does not depend on the length of the hold: across the sweep, its median stays between 0.098 and 0.108 ms for every setting from 4 to 30 ms. Consequently, the release tail stays two orders of magnitude below the timers of Section IV-D.

3) *Impact on Protocol Timers: Retransmission.* On the master-facing captures, neither case carried a TCP retransmission. The longest the master waited under the framework was 20.19 ms for an acknowledgment and 98.55 ms for a response. In a separate test that withheld every acknowledgment of one request, the master repeated the request after 200.8 ms, about ten times the longest wait for an acknowledgment.

Select Validity. The master issues its OPERATE 0.354 ms after the SELECT response reaches it, so the hold on that response consumes part of the select-to-operate timeout. No obfuscated OPERATE was refused, and all 5,280 SELECT and OPERATE exchanges returned a success status.

Coverage. In Figure 6(a), we show the Timing OFF request-to-response time of the read lane against the budget. At 28 ms, the budget covers 99.94% of exchanges, and 17 of 29,040 exceed it. Such a response arrives too late to be held and leaves with its own timing, which gives the largest released READ interval of 78.443 ms. A larger budget covers more of these exchanges at the cost of more delay, as argued in Section IV-D.

C. Limitations

Five limitations bound these results.

For Other Devices. We evaluate one SEL-751A behind one Tofino-1 switch. Our security argument in Section IV-B does not depend on the device, and the three operations of the relay, whose execution times differ, exercise it. However, only a measurement across outstations from several vendors, as Formby et al. and Lin et al. made for the original interval [14], [20], can test the argument for device types directly, and we leave it to future work.

For Adaptive Adversaries. An adaptive adversary learns which operation an exchange carries from when its request reaches the switch, as shown in Section VI-A2. This does not give it the execution time of the relay, and the function code already tells it the operation. However, an operator who wants the released intervals to carry no arrival information at all needs a release that does not depend on the state of the blocker queue, which we leave to future work.

For the Evaluated Attacker. Our classifiers are random forests on two intervals, and the 22 runs are grouped collections within one campaign on one testbed. Their spread is within-campaign variability rather than a confidence interval, and it says nothing about other days or deployments. However, the replacement itself (Section VI-A1) is measured on every exchange and depends on no classifier.

For the Relay-Facing Link. We capture the master-facing link, where the adversary sits. As such, the command hold J and the release toward the relay are configured values rather than measured ones, and we observe neither how many copies of a command reach the relay nor the physical operation that follows.

For Retransmitted Commands. After releasing an OPERATE, the control lane keeps its generation until the next SELECT and drops a retransmission that carries it. A command lost on the relay-facing link is not repaired by its own retransmission, and extending the control lane to forward such a retransmission is future work.

VII. RELATED WORK

Device Fingerprinting. Devices can be identified from what they emit, without access to the payload. Kohno et al. fingerprint a remote host from the skew of its clock in TCP timestamps [28], and GTID identifies a device and its type from the timing signature of its packets [29]. These works define the threat we address, and each of them reads a different observable. Compared to them, we target the interval that the cross-layer response-time method of Formby et al. classifies by, and we replace that interval on the master-facing link.

ICS and Power-System Fingerprinting. Formby et al. brought fingerprinting to control systems with the cross-layer response time and the physical operation time of DNP3 outstations [14], and Gu et al. added device physics as a feature [30]. Pospisil et al. identify PLC models passively from the time delay in their dataflow [17], which shows that execution time still identifies industrial devices under other protocols. The reconnaissance these techniques serve enables false data injection and process-control attacks [24],

[31]. Consequently, the interval our framework replaces is the observable this line of work relies on.

Anti-Reconnaissance in Power Grids. The defenses closest to ours change what the traffic says and which nodes it reaches. DefRec virtualizes physical functions and crafts decoy data for virtual nodes [20], RAINCOAT randomizes data acquisition and injects decoy measurements [32], and a companion testbed evaluates such approaches on grid infrastructure [33]. These approaches are effective against reconnaissance of network and physical configurations. However, they build their protection above the packets an outstation sends, so the execution time of a real device still travels with its answer. Our framework, on the other hand, works below the semantics they reshape and changes when the wire carries that answer, which makes the two complementary.

General Traffic Obfuscation. Traffic morphing transforms the packet-size distribution of one class into another [3], while Dyer et al. show that per-packet padding and fragmentation still leave exploitable features [4]. Tamaraw fixes packet size and rate at a bandwidth cost [8], and WTF-PAD pads inter-packet gaps adaptively [10]. These defenses are effective against the flow-level features they target, and the attacks that follow them have become correspondingly strong [5], [7]. The same size-and-timing channel identifies smart-home devices under encryption, where shaping is the countermeasure [34], and Pacer and NetShaper shape the traffic of a tenant to a schedule that bounds side-channel leakage in the cloud [35], [36]. Pacer and NetShaper each shape from a host or a hypervisor that can add cover traffic, and the smart-home setting assumes an encrypted payload. Compared to these settings, ours carries a plaintext protocol whose leaking feature is one interval inside a single request-response exchange, and no endpoint can be modified to help. Consequently, our framework adds no byte and runs entirely in the network.

Programmable Data-Plane Traffic Transformation. P4 defines the programmable parser and match-action model we use [21]. PIFO defines programmable scheduling at line rate [37], and SP-PIFO approximates it with the strict-priority queues of today’s switches [22], which is the primitive our blocker queues use. Ditto pads packets and injects chaff to a fixed pattern at line rate, and it carries up to 70 Gbps of production traffic on a 100 Gbps link [11]. Minos morphs and schedules traffic on a programmable switch at low cost [12], and Securitas combines fragmentation and insertion across several data planes [13]. NetHide obfuscates topology in the data plane against link-flooding reconnaissance [38]. Closest to our setting, Hu et al. use programmable switches to enhance the security of industrial protocols [39]. These systems reshape which packets a link carries and how large they are. Our framework, on the other hand, addresses one interval inside a transaction, and it holds packets rather than adding or reshaping them.

DNP3 Timing and Security. East et al. catalogue attacks on DNP3 and the absence of encryption that makes its semantics visible [23], and specification-based and state-based intrusion detection has been adapted to DNP3 [25], [40]. These defenses

inspect what a packet says, and none of them changes the timing an adversary sees. On the control path, Formby et al. read the physical operation time from an event timestamp inside the application payload [14], which is a different channel from the packet timing our framework replaces.

Protecting the Framework Itself. Like any in-network defense, the framework adds a switch to what an operator must protect, and an adversary that compromises the switch sees every hold. However, the framework makes light use of the switch. One pipeline program holds the packets of three function codes and forwards all others unchanged, no controller sits in the forwarding path, and the control plane reads back every policy it installs. Checking this program and its policy is a smaller task than hardening the legacy outstations behind it, which often cannot be patched.

VIII. CONCLUSION

Instead of padding packets or adding dummy ones, we propose an in-network framework that holds the packets carrying the timing of an ICS device and releases them at deadlines computed from the arrival of the request. Because neither release depends on anything the outstation does, the execution time of the device reaches neither the request-to-acknowledgment interval nor the cross-layer response time an adversary measures, which is the interval that cross-layer response-time fingerprinting relies on. The framework runs without encryption and leaves the master, the outstation and DNP3 unchanged.

On a physical SEL-751A relay, the released interval settles on the configured 8 ms for every operation, and the interquartile range of READ falls from 2.772 to 0.022 ms. READ and SELECT, which differ only in execution time, become indistinguishable, and a classifier trained before deployment falls from 0.803 balanced accuracy to chance. An adaptive adversary reaches 0.445 by learning when requests reach the switch, not how the relay executes them. The framework adds no frame and no byte, and the median request-to-response time of a poll stays at 28.107 ms, well within every timer of TCP and DNP3.

In future work, we will evaluate the framework on outstations from several vendors, and we will make the release independent of the state of the blocker queue.

OPEN SCIENCE

We reproduce every number and every campaign figure in this paper from the raw captures with one script, in a pinned environment archived with the extraction and analysis code. A hash manifest covers each published artefact, and a publication gate compares a fresh rebuild against what the paper reports, so a discrepancy fails rather than passing silently. We also publish the switch program that ran, with its compilation record. The firmware of the relay and its vendor documentation are not ours to distribute. On acceptance, we release the captures, the analysis pipeline and the switch program under an open license.

ETHICS CONSIDERATIONS

This work required no human subjects and no personal data. We ran every experiment on a laboratory testbed we own, namely one SEL-751A protective relay, one Intel Tofino-1 switch and one master host, on an isolated network with no connection to any operational system. We contacted no production grid, no utility network and no third-party device.

The relay is physical equipment whose outputs can operate a breaker. Every select-before-operate exchange in the evaluation addresses isolated control points through a guarded driver. No control operation in this paper moved a breaker or energized a circuit, and each OPERATE is a protocol exchange the relay accepted and acknowledged.

The framework is defensive. It removes a timing feature that an adversary uses for reconnaissance, and it neither creates a new attack nor discloses a vulnerability in the relay, the switch or the DNP3 protocol.

REFERENCES

- [1] R. M. Lee, M. J. Assante, and T. Conway, “Analysis of the cyber attack on the Ukrainian power grid,” Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016, tLP: White, 18 March 2016.
- [2] R. Langner, “Stuxnet: Dissecting a cyberwarfare weapon,” *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.
- [3] C. V. Wright, S. E. Coull, and F. Monrose, “Traffic morphing: An efficient defense against statistical traffic analysis,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2009.
- [4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, “Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail,” in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2012, pp. 332–346.
- [5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, “Deep fingerprinting: Undermining website fingerprinting defenses with deep learning,” in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2018, pp. 1928–1943.
- [6] T. Wang and I. Goldberg, “Walkie-talkie: An efficient defense against passive website fingerprinting attacks,” in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1375–1390.
- [7] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, “Website fingerprinting at internet scale,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.
- [8] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, “A systematic approach to developing and evaluating website fingerprinting defenses,” in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.
- [9] W. Wang, M. Motani, and V. Srinivasan, “Dependent link padding algorithms for low latency anonymity systems,” in *Proc. 15th ACM Conf. on Computer and Communications Security (CCS)*, 2008.
- [10] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, “Toward an efficient website fingerprinting defense,” in *Proc. European Symp. on Research in Computer Security (ESORICS)*, 2016, pp. 27–46.
- [11] R. Meier, V. Lenders, and L. Vanbever, “Ditto: WAN traffic obfuscation at line rate,” in *Proc. Network and Distributed System Symposium (NDSS)*, 2022.
- [12] Z. Wang *et al.*, “Minos: A lightweight and dynamic defense against traffic analysis in programmable data planes,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2025.
- [13] G. Xie *et al.*, “Securitas: Defending against traffic analysis attacks with flexible in-network obfuscation,” in *Proc. 23rd USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2026.
- [14] D. Formby, P. Srinivasan, A. M. Leonard, J. D. Rogers, and R. A. Beyah, “Who’s in control of your control system? Device fingerprinting for cyber-physical systems,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.
- [15] C. Shen, C. Liu, H. Tan, Z. Wang, D. Xu, and X. Su, “Hybrid-augmented device fingerprinting for intrusion detection in industrial control system networks,” *IEEE Wireless Communications*, vol. 25, no. 6, pp. 26–31, 2018.
- [16] C. M. Ahmed, M. Ochoa, J. Zhou, and A. Mathur, “Scanning the cycle: Timing-based authentication on PLCs,” in *Proc. ACM Asia Conf. on Computer and Communications Security (ASIACCS)*. ACM, 2021, pp. 886–900.
- [17] O. Pospisil and R. Fudjak, “Identifying industry devices via time delay in dataflow,” in *Proc. 13th Int. Conf. on Communication and Network Security (ICCNS)*. ACM, 2023, pp. 173–177.
- [18] IEEE, “IEEE standard for electric power systems communications—distributed network protocol (DNP3),” IEEE Std 1815-2012, 2012, iEEE Std 1815-2012.
- [19] W. Eddy, “Transmission control protocol (TCP),” RFC 9293, Internet Engineering Task Force, 2022.
- [20] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, “DefRec: Establishing physical function virtualization to disrupt reconnaissance of power grids’ cyber-physical infrastructures,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.
- [21] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, “P4: Programming protocol-independent packet processors,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.
- [22] A. G. Alcoz, A. Dietmüller, and L. Vanbever, “SP-PIFO: Approximating push-in first-out behaviors using strict-priority queues,” in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.
- [23] S. East, J. Butts, M. Papa, and S. Sheno, “A taxonomy of attacks on the DNP3 protocol,” in *Critical Infrastructure Protection III (IFIP AICT, Vol. 311)*. Springer, 2009, pp. 67–81.
- [24] Y. Liu, P. Ning, and M. K. Reiter, “False data injection attacks against state estimation in electric power grids,” in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.
- [25] H. Lin, A. J. Slagell, C. Di Martino, Z. Kalbarczyk, and R. K. Iyer, “Adapting Bro into SCADA: Building a specification-based intrusion detection system for the DNP3 protocol,” in *Proc. 8th Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013.
- [26] Schweitzer Engineering Laboratories, *SEL-751A Feeder Protection Relay Instruction Manual*, 2013, pM751A-01-NB, 20130329 printing; settings table, STIME01 select/operate time-out.
- [27] V. Paxson, M. Allman, J. Chu, and M. Sargent, “Computing TCP’s retransmission timer,” RFC 6298, Internet Engineering Task Force, 2011.
- [28] T. Kohno, A. Broido, and K. C. Claffy, “Remote physical device fingerprinting,” *IEEE Trans. on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.
- [29] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, “GTID: A technique for physical device and device type fingerprinting,” *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.
- [30] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, “Fingerprinting for cyber-physical system security: Device physics matters too,” *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, 2018.
- [31] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, “Attacks against process control systems: Risk assessment, detection, and response,” in *Proc. 6th ACM Symp. on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.
- [32] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, “RAINCOAT: Randomization of network communication in power grid cyber infrastructure to mislead attackers,” *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.
- [33] H. Lin, B. Shrestha, and Y.-C. Hu, “Cyber-physical testbed: Case study to evaluate anti-reconnaissance approaches on power grids’ cyber-physical infrastructures,” in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.
- [34] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, “Keeping the smart home private with smart(er) IoT traffic shaping,” *Proceedings on Privacy Enhancing Technologies (PoPETs)*, vol. 2019, no. 3, pp. 128–148, 2019.
- [35] A. Mehta, M. Alzayat, R. De Viti, B. B. Brandenburg, P. Druschel, and D. Garg, “Pacer: Comprehensive network side-channel mitigation in the cloud,” in *Proc. 31st USENIX Security Symposium*, 2022.
- [36] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, “NetShaper: A differentially private network side-channel mitigation system,” in *Proc. 33rd USENIX Security Symposium*, 2024.
- [37] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, A. Agrawal, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, “Programmable packet scheduling at line rate,” in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.
- [38] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, “NetHide: Secure and practical network topology obfuscation,” in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.
- [39] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, “Industrial network protocol security enhancement using programmable switches,” in *Proc. IEEE Int. Conf. on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.
- [40] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, “Modbus/DNP3 state-based intrusion detection system,” in *Proc. 24th IEEE Int. Conf. on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736.